

Chapter 16 - Native SQL in Hibernate

1. What is Native SQL?

Native SQL allows you to execute database-specific SQL directly through Hibernate or JPA. It is useful when HQL/JPQL cannot express a query efficiently or when database-specific features are required.

Feature	HQL / JPQL	Native SQL
Query Target	Entities	Database tables
Portability	High	Depends on database
Database Functions	Limited	Full database support
Result	Entities / Scalars	Entities, DTOs or Scalars

2. Execution Flow

```
Application
|
createNativeQuery()
|
Native SQL
|
Database
|
Result Set
|
Entity / DTO / Scalar
```

3. Basic Example

```
String sql =
"SELECT * FROM EMPLOYEE WHERE DEPT=?";

List<Employee> list =
entityManager
.createNativeQuery(sql, Employee.class)
.setParameter(1, "IT")
.getResultList();
```

4. Scalar Results

When only selected columns are required, a native query can return scalar values instead of entities.

```
SELECT EMP_NAME, SALARY
FROM EMPLOYEE
```

5. DTO Mapping

Instead of returning entities, native query results can be mapped to DTOs using JPA result set mappings or provider-specific features.

6. Stored Procedures

Hibernate and JPA support calling stored procedures using `StoredProcedureQuery` or provider-specific APIs when required.

7. Pagination

Pagination can still be applied using `setFirstResult()` and `setMaxResults()` on the query object.

8. Named Native Queries

Frequently used native queries can be declared with `@NamedNativeQuery` and reused throughout the application.

9. Best Practices

- Prefer HQL/JPQL for portable queries.
- Use Native SQL only when necessary.
- Avoid database-specific SQL unless the application is tied to one database.
- Map results carefully to entities or DTOs.
- Use parameters instead of string concatenation.

10. Interview Questions

1. What is Native SQL? 2. Difference between HQL and Native SQL? 3. When would you choose Native SQL? 4. Can Native SQL return entity objects? 5. How do you call a stored procedure from JPA/Hibernate?